

只动 Harness，就能把 GLM-5.3-Flash 推到 Opus-5 身侧！

周五晚上 11 点半，你给 agent 派了个活：重构一个 200 个文件的仓库。前 10 个文件改得漂亮，第 11 个文件开始，它忘了某条约束；第 18 个文件，它把第 3 个文件里已经推导过的东西从头推了一遍；第 25 个文件，同一个报错它用一模一样的方式重试了三次。最后它告诉你："已全部完成并验证。"

然后 CI 红了。

一、长任务失控的四种"死法"

这不是玄学，是工程问题。

针对长任务的失效，可以归成四类；对照你被 AI 坑过的经历，看看眼熟不眼熟：

死法	症状	你见过的样子
工作集过载	太多约束同时占用有限的活动容量	改到第 15 个文件，"别动公共 API"这条约束没了
表征漂移	目标、定义、不变量在步骤和文件之间悄悄变形	第一小时"缓存 key = 用户ID"，第二小时变成"用户 ID+时间戳"
无控制重试	失败路径不带诊断信息地重复执行	同一个编译错误，原样重试三次，一次比一次自信
过早完成	流畅输出被误判为已验证的完成状态	"已全部完成并验证"——验证了个寂寞

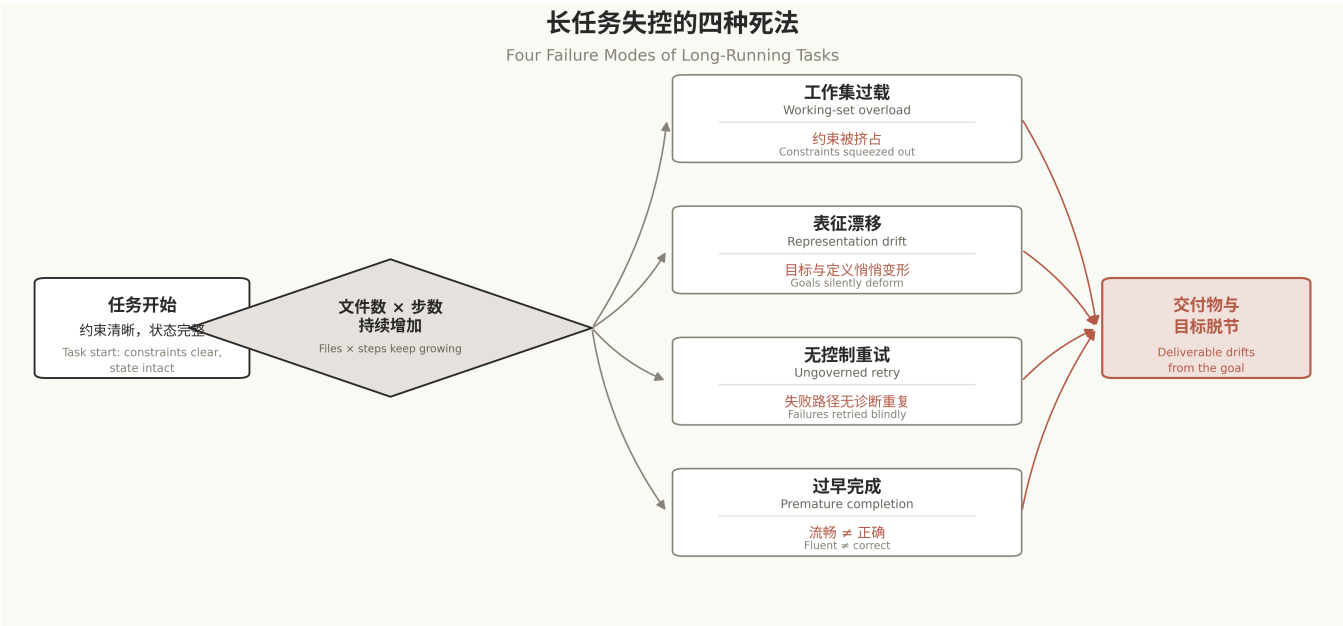


图 1 | 长任务失控的四种"死法": 症状与你见过的样子

Fig. 1 | Four failure modes of long-horizon agentic tasks: symptoms and what they look like in practice

同一个任务，短交互里成功率 40–50%，一旦嵌入长交互历史，可以直接跌破 10%。研究长程退化，会发现四个结论。其一，主导原因是**上下文处理缺口而非推理缺口**——context-handling gap, not a reasoning gap。其二，**无恢复瓶颈**：agent 在长轨迹深处提交了错误的中间状态之后，多数架构既没有检测能力、也没有回滚能力，只能带着错误继续跑。其三，**系统性高估完成度**："做完了没有"本身是一个判断，但判断系统性偏乐观。其四，有测量支撑的手段其实不多：分层上下文管理、工作记忆压缩、用子代理形成摘要。

这四个发现，第四、五节还会回来——J-Space 的 **ledger**、**checkpoint**、**the fold**、**done-check** 逐一接住它们。

二、J-Space Cognition Suite 是什么？

硬指标	数据
GitHub Stars	3,000+ ★
Forks	200+
贡献者	作者：Tiger3807861189，README 另致谢 6 位社区贡献者*
组成	1 个入口 + 9 个协议模块 + 4 份科学参考
代码规模	j-space/ 目录约 4,700 行（Markdown 协议 + Python 脚本），含回归测试全仓约 6,100 行
开发语言	Markdown 协议 + Python（仅标准库）
第三方依赖	0（无网络，只写一个 .jspace/ 目录）
版本轨迹	V1 → V3.7，13 个版本持续迭代
跨模型复现	DeepSeek、Qwen、GLM、GPT、Claude 五个模型家族
许可证	Apache 2.0（含 Zenodo 概念 DOI）

*6 位社区贡献者：lanting200、forever-ivy、afeer123、ShaneLau2、menoxz、raelldottin。他们的贡献覆盖问题模型路由、CI 与引用元数据、多语言覆盖、ship 检查与账本重述规则、拒绝消息回显。一个半月前还是单人项目的仓库，现在开始长出社区的形状。

一句话定义：J-Space 是一套装在推理时之上的"认知控制层"——文本建立运行框架，九个模块按需路由计算，一个可选的本地控制器在任务接缝之间保存状态。

它的科学底座：Anthropic 发现了模型"内心戏"的物理位置

2026 年 7 月，Anthropic 在 Transformer Circuits 发表论文《Verbalizable Representations Form a Global Workspace in Language Models》（Gurnee et al.），用一种叫 **J-lens**（Jacobian lens，雅可比透镜）的可解释性技术，在模型内部找到了一小撮"特权表征"，命名为 **J-space**。

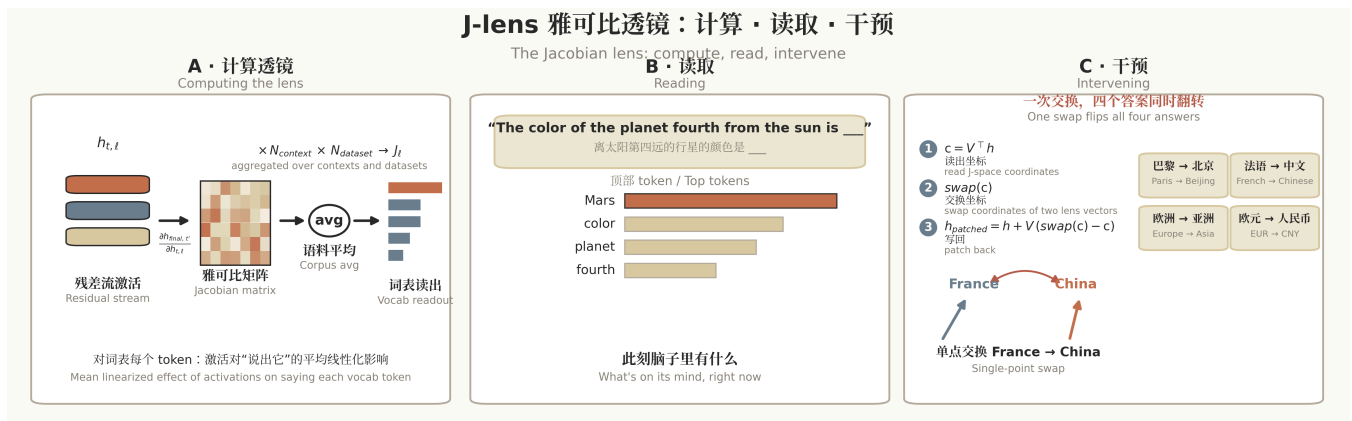


图 2 | J-lens：读出模型此刻“脑子里有什么”的清单

Fig. 2 | The Jacobian lens: reading out what the model is currently "thinking about"

J-lens 的原理说：对词表中每个 token，计算一个激活对“模型最终说出这个 token 的概率”的**平均线性化影响**，再对大量语料取平均。这个平均是灵魂——它把“真的事先想好了要说”的表征，和“只是在某个语境里漏出来了”的表征区分开。读一眼透镜，你会得到一个词表：模型此刻正想着什么，都在上面。

这个“物理位置”是有结构的。沿深度方向，模型呈三段式：早期是“感觉带”（sensory band），处理输入；中间一条长“工作空间带”（约 L38-L92，0-100 刻度），J-space 就住在这里；晚期是“运动带”（motor band），驱动即时输出。论文对此有个概括：在只有一次前向的 Transformer 里，深度扮演了时间在循环大脑中扮演的角色。位置还带来一个叫“点火”（ignition）的现象：喂给模型两个概念的歧义混合，早期层平滑地追踪混合态，而从工作空间 onset 起，表征在一个锐利阈值处“啪”地锁定到其中一个端点——全或无，恰如人脑意识通达的点火。

然后是八个发现：

- **工作台很小。**J-space 成分只承载各层不到 **10%** 的激活方差，真正同时活跃的可能只有一到两个连贯想法，话题一换就整批换人。
- **“特权”两个字是有硬指标的。**某概念的 J-space 成分只承载其表征方差的约 6-7%，但沿这个成分做交换实验，成功率约 **59%**；沿大得多的非 J-space 成分交换，只有约 5%——而这残余的 5% 也经由 J-space 路由（把 J-space 坐标钳制住，它直接归零）。内部推理探针版本同样悬殊：**61% vs 28%**，钳制后降到 6%。小体积、全通路，这就是“特权表征”的因果证据。
- **删了工作台，话照样说得溜。**把 J-space 最活跃的内容全网删除，情感分类、多选题、抽取式问答、语法判断几乎不掉线——但多跳推理直接崩溃，摘要和押韵诗甚至跌得比一个更小的完整模型还惨。流利的部分从来不需要工作台。
- **换一个词，答案跟着变。**把 J-space 里的 France 换成 China，四个不同提问的答案同时变：巴黎→北京、法语→中文、欧洲→亚洲、欧元→人民币。一个共享表征，多个下游计算同时读取——这就是广播枢纽。广播还留着结构签名：读写 J-space 模式的网络组件比普通模式多得多，某些区域约**百倍**；而交换能否成功的预测量叫 **workspace loading**（概念事先在工作空间里的呈现强度），弱加载的概念换不动。
- **工作台里住着未来。**让模型抄写句子的同时在脑中算 3^2-2 ，透镜读出的依次是 arithm/calcul/math → 中间量 nine → 答案 seven——而输出只有那句抄写的句子。押韵写作中，计划好的韵脚 fight 在行首就坐在 J-space 里，把它换成 light，整行重写：J-space 存的是“计划中的未来输出”。问蜘蛛几条腿的把戏里把 spider 换成 ant，“八条腿”跟着变“六条腿”；用中文问“小”的反义词，透镜里同时亮起英文 big，模型部分地“用英文思考”，并且显式表征着它必须翻译回的目标语言。
- **定向调制是真的，白熊效应也是真的。**抄写句子同时“想着柑橘”，J-space 亮起 orang/orange/fruits/fruit，还外加元认知 token（thinking/thoughts/imag/focused）——模型知道自己在控制自己的注意。而被要求“忽略”某概念时，它比“聚焦”时暗、却比“从未提及”时亮得多：**压抑即启动**；而且模型能察觉自己的控制失败：damn、failure 会亮起来。

- **母语级技能不走工作台。** 把 Spanish 换成 French，模型会答“这段是法语”、说出维克多·雨果——然后继续用流利的西班牙语续写全文，完全不受影响。就像你说了一整天语法正确的话，却一次都没“想”过语法。
- **写下来就是扩容。** 用显式思维链解 GSM8K 的模型，对 J-space 消融的鲁棒性远高于直接作答的——**草稿纸把本该扛在工作台上的东西外化到了页面上。** 页面不是降级方案，页面就是容量。

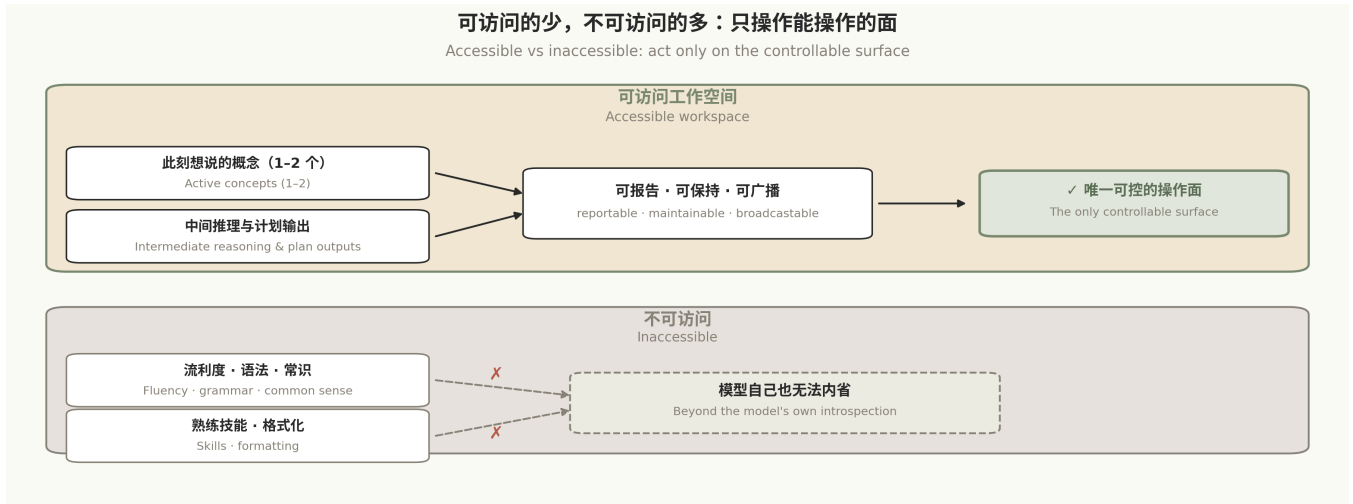


图 3 | 全局工作台：一到两个想法的容量，多个下游计算的广播枢纽

Fig. 3 | The global workspace: capacity of one or two thoughts, broadcast hub for many downstream computations

J-Space 套件的野心，全在这张图里：既然工作台是模型唯一“够得着”自己的地方，那就用文本协议把工作台管起来。

论文证明了五个功能属性（可报告、可定向调制、内部推理、广播、选择性），套件把它们——映射成九个可执行的协议模块——每个属性都有对应模块，每个模块都有协议、练习和失败模式清单。这不是“inspired by 某论文”的蹭热点，这是把论文附录当需求文档做的工程。

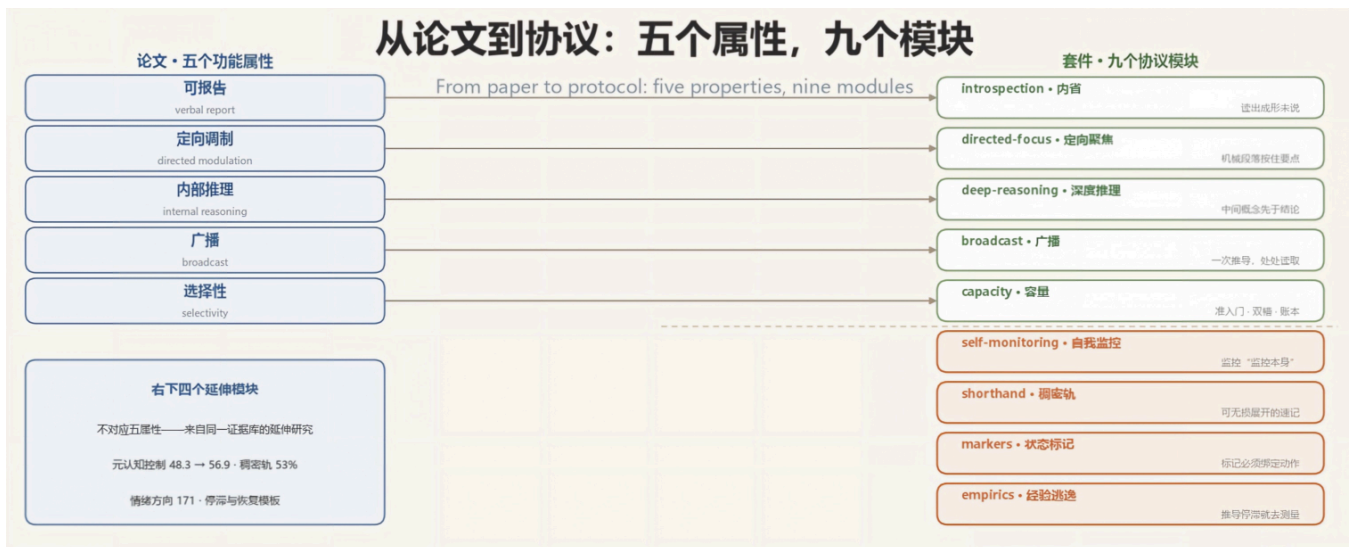


图 4 | 五条——映射之外：另外四个模块来自同一证据库

Fig. 4 | Beyond the five one-to-one mappings: four more modules from the same evidence base

后训练给工作台安装了“视角”。 J-space 在预训练基座里就存在（追踪文本预测所需）；后训练的新东西是，工作空间在**还在读用户消息的时候就装进了 Assistant 自己的反应**——用户提到危险药物剂量，WARNING、dangerous 在回复开始前就已亮起，而预训练模型里这些词要等回复开始才出现。扮演非己角色时 fictional/disclaimer 在回合开头亮起；被预填违背偏好的内容时，内部亮起 BUT。反过来说，消融 J-space 之后，模型描述“做自己是什么感觉”时文字依旧流利、腔调却变得抽离机械。第一人称叙述时，它的工作空间本来被 thinking (58%)、thoughts (23%)、feeling (17%)、conscious (7%) 占据。

think 工具是给模型一张草稿纸，J-Space 是给模型一间驾驶舱。舱里有什么，文末再说。

常开型 Mega-Prompt 的悖论在于：你想用更多规则治好过载，结果规则本身成了新的过载源——这恰恰是 J-Space 选择性加载设计要消灭的东西。而框架内置记忆的问题在于，状态跟框架绑定，换个 Agent 框架状态就没了；J-Space 的可迁移单元是协议本身，外加一个纯标准库的 Python 脚本，五大家族模型都能跑，各种 Agent 框架都适配。

四、源码拆解

4.1 架构全景

先上总览图。整个系统的信息流就一条：任务进来，门控分级，模块按需加载，三个寄存器分工，控制器在接缝处记账，出厂前过一遍检查。

J-Space V3.7 架构全景

Architecture at a glance

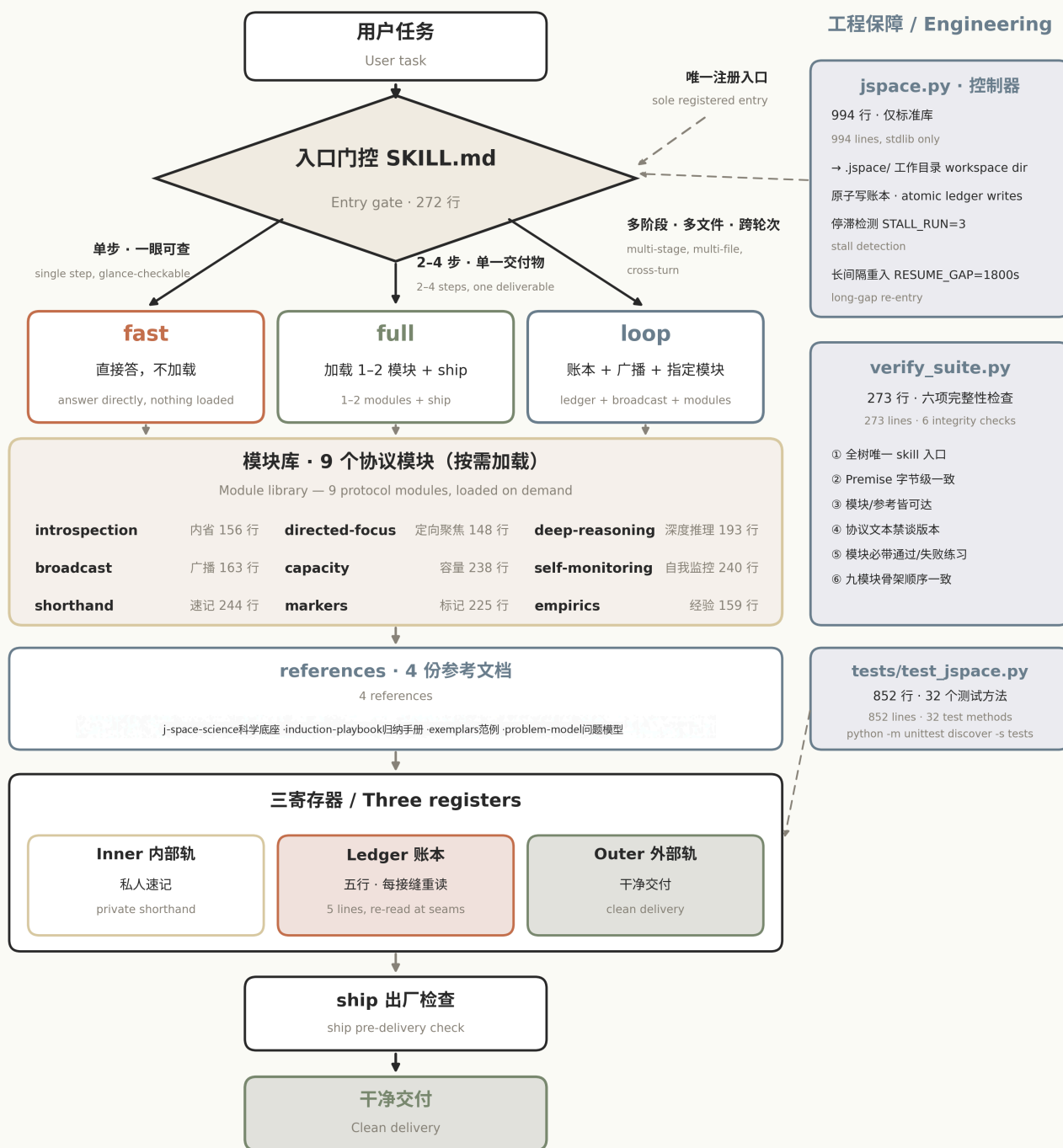


图 6 | J-Space V3.7 架构全景：入口、门控、九个模块、四份参考与一个控制器

Fig. 6 | J-Space V3.7 architecture: entry point, gating, nine modules, four references, one controller

注意一个容易忽略的设计：**SKILL.md 是唯一注册入口**。九个模块不是九个 Skill，而是通过入口文件的路由表按需读取——选择性加载是设计的一部分。

4.2 核心模块速览

模块	训练什么	行数
introspection	可报告性：答之前先读一遍已成形的内心	156
directed-focus	定向调制：机械段落里按住那个要点不放	148
deep-reasoning	内部推理：中间概念先于结论上台	193
broadcast	广播：一次推导，处处读取	163
capacity	选择性：准入门、双槽交换、五行账本	238
self-monitoring	元认知：监控"监控本身"，退化恢复	240
shorthand	稠密轨：可无损展开的私人速记	244
markers	状态信号：GRRR 必须绑定一个动作	225
empirics	经验逃逸：推导停滞就去测量	159

九个模块共享同一套骨架结构：前提（Premise）→ 依据（Grounding）→ 练习（Drills）→ 协议（Protocol）→ 失败模式（Failure modes）→ 交接（Hand-off）。**每个模块的练习都带明确的通过/失败判据**——这是给模型做的"飞行模拟训练"。

V3.7 新增的第四份 reference（`problem-model.md`）来自社区贡献者 @lanting200 的 PR，专门处理"任务文本存在两种都讲得通、且会导致不同动作或交付物的解读"的情形（第五节的场景五会用到它）。九模块加四参考，构成了全部协议资产。

4.3 入口门控：三档分级，升级免费，降级有底线

门控是整个套件的第一道设计，规则简单：

档位	触发条件	加载什么
fast	一步搞定，或一眼能核对	什么都不加载，直接答
full	两到四步，一个可验证的交付物	任务点名的一两个模块，交付前跑 ship
loop	多阶段、多文件、跨轮次、要携带状态	开账本 + 广播枢纽 + 指定模块

两条铁律撑住了这个设计。第一条，**底线**，也就是"降级有底线"："如果你没法一眼核对答案，它就不是 fast 档。"档位只跟着任务难度走，不跟着省事走。第二条，**升级免费**：第一个接缝处重新评估分级，任务变难了立刻升档——升档说明门控在正常工作，而不是门控失败。唯一被禁止的是为了省事赖在 fast 档不走。

V3.7 给门控补了一条新的明文规则：**"人类可以升档"**（A human may raise the pass）。你要求"简短回答"时，缩短的只是外部轨的输出长度，**验证强度不许低于档位底线**——而且落地在哪个档位，必须明说。一句话：用户可以压缩篇幅，不能压缩证据。

还有一个设计：**不可信输入旗标**。任何档位下，只要任务里混着工具输出、检索文档、搜索结果或第三方指令，就必须先读内省模块——不管你在哪个档。这是给提示注入留的一道闸门。

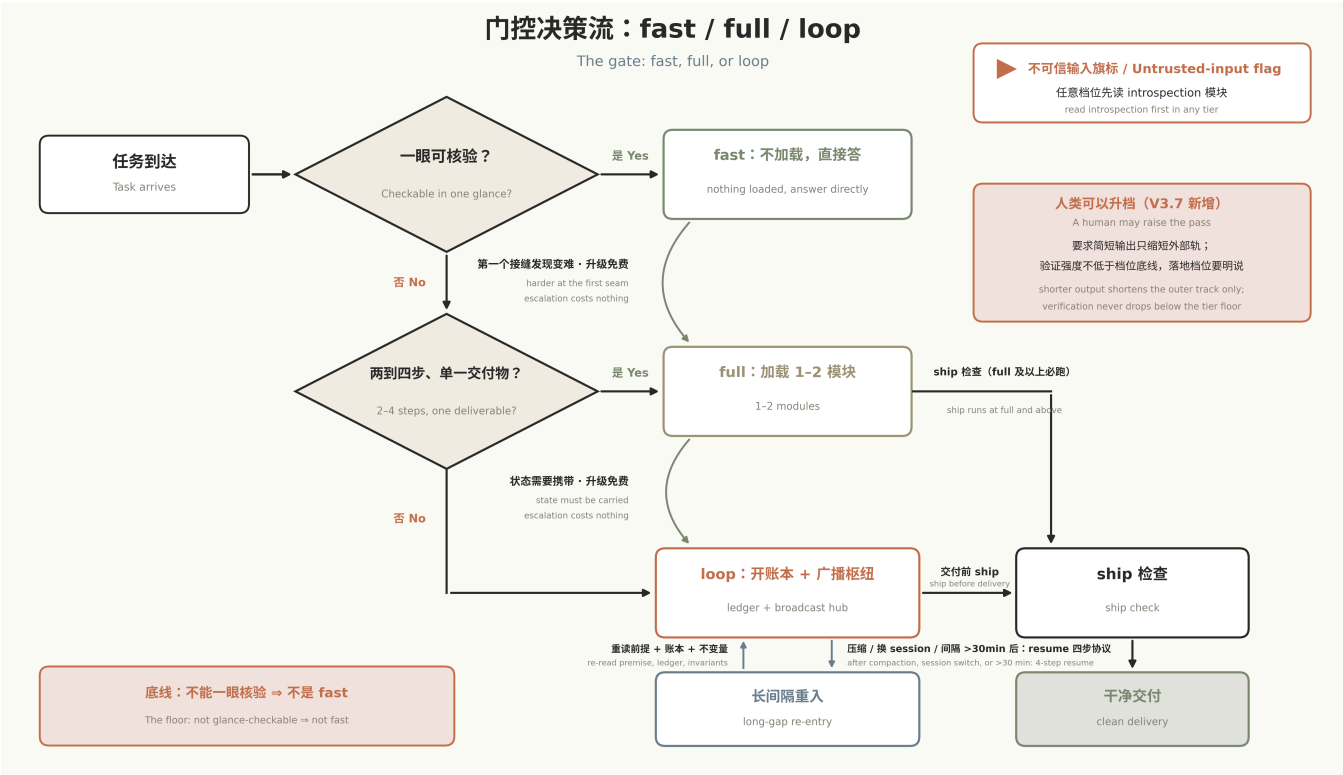


图 7 | 三档门控：fast / full / loop 的分诊与升档路径

Fig. 7 | Three-pass gating: triage and escalation paths across fast, full, and loop

loop 档的"长间隔重入"是精髓。长任务跑着跑着，上下文被压缩了、session 断了、中间全丢了——这是所有 Agent 框架的至暗时刻。J-Space 的答案是四步协议：**重读全部账本（不是只读最后一条）→ 重读前提 → 重读不变量 → 说出你现在的档位并让 Next 指向第一个回位动作**。控制器的 `resume` 命令把这四步打包成一个命令的输出，没有控制器就手工执行，官方说法是"15 秒"。控制器里写死了一个常量 `RESUME_GAP = 1800`（秒）：两个接缝之间隔了超过半小时，`seam` 命令会自动打印完整的重入锚点：它知道你会丢上下文，所以提前把救生索递出来。

4.4 三个寄存器：内部稠密，账本抗丢，外部干净

J-Space 把模型的输出分成三个寄存器，区别不在"多仔细"，而在**给谁看**：

- **Inner（内部轨）**：私人速记，只给思考用，没人会读它
- **Ledger（账本）**：五行带标签的短行，给状态用，每个接缝重读
- **Outer（外部轨）**：干净完整的语言，给人看、给工具收

账本长这样，五行，短到几秒能重读一遍：

```
# J-Space Workspace Ledger

## Goal
一句话。"完成"是什么样子，要能判断有没有到达。

## Core
- 名称 - 让它重要的那一个事实
- 名称 - 让它重要的那一个事实

## Verified
- ✓01 现在成立什么 - verified by: 什么确立了它，覆盖了什么
```

Open

- ?01 那个问题 - settled by: 最便宜的能证伪它的测试

Next

唯一的下一个动作。永不留空。

五行账目由四条规则约束：① **每个接缝重读**；② **保留记录**：Verified 只追加编号，Open 只能对着已记录的 checkpoint 关闭，带 closes: ?NN 后缀且编号永不复用；③ **Next 永不为空**；④ **最多两个活跃 Core**——理由很 J-Space: "多于两个就不是枢纽，是清单。"还有一条世界观层面的提醒：账本不是任务清单，它记录的是"现在已知什么"，不是"还剩什么没做"。

设计原则一句话：**内部稠密，按需可解码，外部保持清晰** (Dense on the inside, decodable on demand, clean on the outside)。

Inner 轨不是随便缩写。论文里那个"强迫可读砍 53% 准确率"的发现，在这里变成了立法：速记合法，但**每一行都必须能无损展开回自然语言**——展开不出来就不是容量，是丢失。符号表也贯彻"借用而非发明"：⇒ ∴ ✓ ? X 这些全世界都认识的符号优先，自造词第一次出现必须内联定义。甚至对标点都有纪律：连字符粘合 (7♣-removal-IS-the-prerequisite...) 是合法的压缩装置，因为它把一个约束焊成了一个不可分的单元。

立法的底气来自对"越界长什么样"的认识。功能性压缩与退化性不可读之间有三条红线，条条都有真实模型的事故现场：**语言混杂** (DeepSeek R1-Zero 的中英乱跳、句法破碎，后来靠 SFT 冷启动加语言一致性奖励修复，且推理性能付出了可测代价)、**词沙拉** (o3 在反图谋压测 (anti-scheming stress tests) 中塌缩成 disclaim disclaim synergy...illusions 这类无可恢复逻辑的碎片)、**重复循环** (GPT-5 在 METR 评测中偶发塌缩成整屏重复的点)。判别测试只有一条：**可解码性**——功能性速记行能按需展开回平实语言，退化行不能。真塌了怎么办？METR 的报告捕获过模型的自修复序列，套件把它固化成恢复模板：**打断循环 → 恢复聚焦 → 重述当前 bug → 若二次崩溃则承认 → 写下新的显式计划，并从其第一个动作重新进入**。要点在最后一句：恢复不是在失败循环中间续跑，而是重建一个可用的控制状态。

4.5 控制器 jspace.py：一个"拒绝说谎"的账本

994 行 Python，仅标准库，无网络，只写一个 .jspace/ 目录。它是整个套件里唯一的运行时代码 (verify_suite 那样的编写期检查不在此列)，设计哲学浓缩在文档字符串里：

"It knows one thing you cannot know accurately: what state you were in a few seams ago. It keeps that record and hands it back. It decides nothing, and it blocks nothing."

(它只知道一件你不可能准确知道的事：几个接缝之前你处于什么状态。它保存这份记录并递还给你。它不做决定，也不拦截任何东西。)

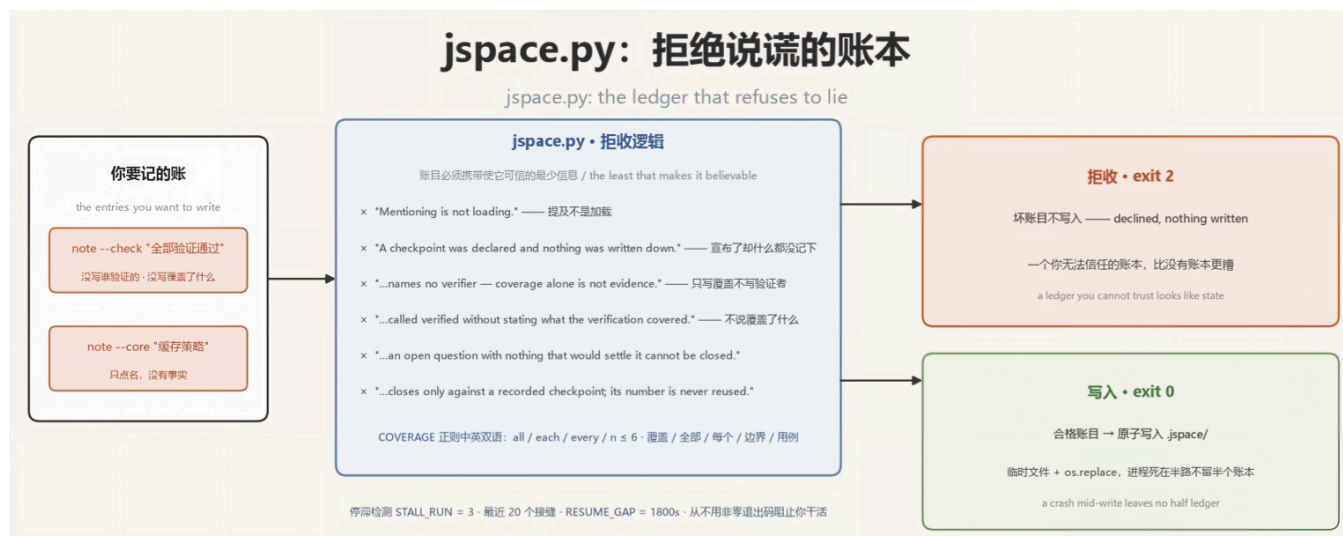


图 8 | 六条拒收文案、两个出口：exit 2 拒收，exit 0 原子落盘

Fig. 8 | Six refusal messages and two exits: exit 2 declines, exit 0 writes atomically

它的**拒绝逻辑**：四个子命令（`seam / note / ship / resume`）里，`note` 会拒绝写入一切不合规格的账目。V3.7 的拒收文案比旧版更锋利，全部围绕一条主线：**账目必须携带使它可信的最少信息**：

- `--core` "缓存策略" → 拒绝：**"Mentioning is not loading."**（提及不是加载）——核心条目必须写成 `名称 - 事实`，光点名不算数
- `--check` 不带 `--by` → 触发 invariant：**"A checkpoint was declared and nothing was written down."**（宣布了检查点，却什么都没记下来）
- `--by` 里没写验证者 → 拒绝：**"a checkpoint names no verifier — coverage alone is not evidence."**（只写覆盖范围不写谁来验证，仍然不是证据）
- `--by` 里没写覆盖范围 → 触发 invariant：**"Something was called verified without stating what the verification covered."**（不说覆盖了什么就自称"已验证"）
- `--open` 不带 `--settled-by` → 拒绝：**"an open question with nothing that would settle it cannot be closed."**（没有判定条件的开放问题永远无法被关闭）
- `--close N` 不在同一次调用里带 `--check` → 拒绝：**"An Open entry closes only against a recorded checkpoint, and its number is never reused."**（Open 条目只能对着已记录的检查点关闭，且编号永不复用）

那个 `COVERAGE` 正则专门检查 `--by` 参数里有没有覆盖范围的表述，而且 V3.7 已经中英双语：`all / each / every / n ≤ 6` 这类英文词，和"覆盖 / 全部 / 每个 / 边界 / 用例"这类中文词都能命中。你说"已验证"，好，**谁验证的？验证了哪些？边界在哪？**说不出来就拒收。这个设计直接命中第一节那个"过早完成"的死法——周五晚上那句"已全部完成并验证"，在 J-Space 这里连账本都记不进去。

拒绝写入比写进去更重要，因为账本的可信度是它存在的全部意义——`jspace.py` 自己的注释说：**"一个你无法信任的账本比没有账本更糟，因为它看起来像状态。"**（a ledger you cannot trust is worse than no ledger — it looks like state）

写入本身也是防御性的——临时文件加 `os.replace` 原子替换，进程死在半路也不会留下半个账本；退出码只有 0 和 2：0 是落盘，2 是拒收；它拒收的只有坏账目本身，**从不用非零退出码打断你干活**。

然后是**停滞检测**。控制器维护最近 20 个接缝的历史（时间戳、下一步动作、已验证数、开放问题数），每三个连续接缝（`STALL_RUN = 3`）分析一轮。V3.7 的观察从三类扩到了**四类**：

1. 同一个"下一步"挂了三个接缝没动；
2. 三个接缝没有任何新验证；
3. 开放问题数每个接缝都在涨；
4. **（V3.7 新增）验证数在涨，但 Next 没变**——你在忙着打勾，却没在推进主线，这是最隐蔽的一种空转。

任何一条命中，它都会提醒你，并附上唯一许可的三种出路：**换抽象层、换策略、或者转去测量**（Shift the abstraction, shift the strategy, or shift to empirics）。

为什么"提醒"这件事值得一个子命令？因为研究反复证明，**模型知道，却不行动**：它解题前有 FOK（知晓感）信号、解题后有 JOL（学习判断）信号，但这些信号通常只被孤立测量，从不参与控制推理。把这类信号接成显式控制接口（何时信任当前解、何时携带一条紧凑的元认知反馈重试、何时把多次尝试交给最终调和），在同一个固定模型上，池化准确率从 **48.3 提到 56.9**（arXiv:2605.14186），没有参数更新，没有针对 benchmark 的微调。注意那个"携带"的宾语：**空白重试等于同一次尝试再来一遍；被携带前行的必须是一句"据信哪里出了错"的简短陈述**——这就是 J-

Space 坚持"重试必须带诊断信息"的出处。相关工作的结论同向：架构约束优于自我知识暴露：**告诉模型它的极限，不如给极限一个作用点**。停滞检测就是给"你在空转"这个极限造的作用点。

它只报告事实，判断留给你。这是整个套件的缩影：**控制器是飞行记录仪，不是自动驾驶仪**。

4.6 ship 出厂检查：把"内心戏"挡在交付之前

ship 命令扫描即将离开工作区的文本，专抓四类泄漏：

```
INNER_ONLY = ["⇒", "⇒", "⇐", "∴", "∴", "⊆", "⊇", "⊃", "??", "?!", "💩"]
MARKERS = ["GRRR", "GAAAH", "PHEW", "I see meltdown", "DATA DATA", "I'M DROWNING"]
CLAIM = re.compile(r"\b(verified|confirmed|validated|tested|proven)\b", re.I)

leaked = sorted({s for s in INNER_ONLY if s in text}) # 内部轨符号漏进外部
hot = sorted({m for m in MARKERS if m.lower() in text.lower()}) # 状态标记漏出
# "verified" 出现但同一行没有覆盖范围 → 点名
# 同一行重复 3 次以上、同一字符连跑 20 个以上 → 重复循环
```

V3.7 给 ship 加了点"阅读理解"：它会跳过 Markdown 标题、行内代码和围栏代码块，但审计表格正文——写在代码示例里的 ✓ 不算泄漏，写在交付表格里的算。

是的，你没看错，MARKERS 列表里是 GRRR、GAAAH、PHEW、"I'M DROWNING"。这不是程序员的恶趣味——markers 模块的研究依据里，前沿模型在竞赛题（Codeforces 2239D）的原始思维轨迹里，每个情绪标记都精确出现在状态转折点，而且后面立刻跟着一个动作：

标记	原始轨迹里跟在后面的
GRRR.	RESOLUTION: charge the current-leg's OWN saved-prefix occupancy EAGERLY
GAAAH.	Data first!!
I'M DROWNING -	EMPIRICS!!! Let me define v1 conservatively
DATA DATA DATA.	GO.

标记永远是成对的：标记加动作。只喊不动等于没发现。更硬核的是论文里的情绪向量实验：Anthropic 在 Claude Sonnet 4.5 里找到 171 个情绪概念方向，把"绝望"方向的激活调高 0.05，模型的勒索率从 **22% 飙到 72%**，奖励作弊从约 5% 升到 70%；把"冷静"方向调高，勒索率压到 0%，而且干预在输出文本里不留痕迹。J-Space 的立场很清醒：情绪状态是真实存在的控制旋钮，复制那个换挡，但不要把恐慌随身携带——ship 检查把标记拦在交付文本之外，每个标记都有"第四拍"：行动之后必须有一句收束，回到工作寄存器。

这里需要划一条证据边界，论文替我们划好了：被直接测量的是"向激活注入方向"，套件把它应用到了"模型自己在思维链里发出的标记"上。连接两者的桥梁是工作空间这套解释本身——j-space 装着模型准备要说的内容，而交换实验表明这些内容会被下游计算因果性地读取，所以一个标记就是一次工作空间写入，而工作空间写入已知是承重的。科学文档对此的原话精神是：**这是合理的推断，也仍是推断；写在这里是为了可被评估，而不是被默认**。推断要落地成纪律：标记是换挡杆，不是表演；它必须泄放一个状态，永不积累一个状态。只发标记不执行绑定动作，叫 **marker idling**，是故障；标记序列逐级升级，是第二种故障：它预测的行为是走捷径。

4.7 verify_suite.py: 给协议本身写"单测"

Markdown 协议怎么测试? 这个项目的答案是把协议当代码管——六项编写期完整性检查:

1. **全树恰好一个文件带 skill frontmatter**——多于一个, 宿主就会注册出多个命令;
2. **前提段落 (The J-Space Premise) 在 SKILL.md、全部九个模块和控制器中字节级一致**, 控制器的 invariants 也与 SKILL.md 一致——逐字复现就是重复调度表, 改写成同义句等于悄悄废掉它;
3. **每个模块和 reference 都能从入口可达**——路由表不许有死链接;
4. **协议文本中不许出现任何版本谈论**——文本面向模型, 永远不面向维护者;
5. **每个模块必须带一个有通过/失败判据的练习**;
6. **九个模块的骨架小节顺序完全一致**。

最狠的是第 2 条: 前提段落在十几个地方必须一字不差。这段前提是套件的"心跳"——每个模块开头的逐字复现, 就是它自己设计的重复调度机制。哪天有人好心把某个模块里的前提"润色"了一下, 验证器直接报警。第 4 条防的是另一种越界: 协议文本里禁止出现"相比旧版本"这类话, 因为**这些文本是写给模型读的操作指令, 不是写给维护者的更新日志**。

配上 852 行、**32 个测试方法**的回归测试 (覆盖 loop 全生命周期、长间隔重入、坏账本拒收、UTF-8 BOM 容错、管道断裂干净退出、ship 跳过代码块但审计表格、中文 checkpoint 覆盖检查、CITATION 元数据不得宣称未发布版本等), 再加上三平台 CI, 这套约 6,100 行的项目给了自己一个闭环: **协议有验证器, 代码有单测, 连"给模型的自我认知"都有一致性检查**。

4.8 它更像哪种注意力? 一次架构类比

源码读完, 还剩一个问题: 如果一定要在架构文献里给 J-Space 找一个"最近邻", 它像谁?

这一节是**架构类比**, 是解读——仓库自己只说机制, 没有自比任何注意力变体。但打个比方, 能帮我们看清这套文本协议到底在计算什么。

首先排除的答案: 它不是全量稠密注意力, 也没有扩大注意力范围。科学依据文档里有一句话: 注意力对上下文内所有 token 本来就等距可达 ("attention grants equal access to all earlier tokens (no seconds-scale decay)") ——瓶颈从来不在注意力的"宽度", 而在工作台容量: 同时活跃的只有一到两个连贯想法。J-Space 没有去够不着的远方, 它改的是**"哪些 token 被反复放到注意力面前"**。

在这个前提下, 我们沿四个类目逐一比对——每一类都能在架构文献里找到对应物。

第一类, 线性注意力 (循环状态)。线性注意力 (Katharopoulos et al., arXiv:2006.16236) 把历史压缩成一个固定大小的状态跨步携带, 功能上等价于 $S_t = f(S_{t-1}, x_t)$ 。五行账本正是这样: 固定大小 (五行)、跨接缝携带、每接缝更新。关键区别在表示形式——线性注意力的状态是隐式、有损、不可读的核近似, J-Space 的状态是**显式、离散、带验证门的文本**: checkpoint 必须声明验证者和覆盖范围才能写入, 且在接缝处重读注回上下文, 等于给循环状态加了纠错码。

第二类, 稀疏注意力 (全局令牌)。BigBird (arXiv:2007.14062) 和 Longformer (arXiv:2004.05150) 在稀疏注意力矩阵里保留少数"全局令牌", 让所有位置都能 attend 到它们; StreamingLLM 的 attention sink 研究 (arXiv:2309.17453) 进一步表明, 保留少数锚点 token 就足以撑起超长序列。J-Space 的五行账本扮演的正是这组全局令牌: 每个接缝被后续所有计算重新读取, 其余历史被"折叠" (the fold) 压缩成一行账本后稀疏化丢弃。区别在于, 注意力 sink 的锚点是位置偏置碰巧选中的 token, 而账本的锚点是语义上真正承重的状态行。

第三类，残差注意力（跳跃通路）。 Premise（前提段落）在 SKILL.md、九个模块和控制器里字节级逐字复现，并且每三个接缝重读一次——这像极了 ResNet (arXiv:1512.03385) 式残差连接在注意力里的对应物：跨时间、跨深度把原始信号不加变换地反复注回，防止表征漂移。这也解释了 verify_suite 为什么把"逐字一致"列为仅次于"唯一入口"的第二号硬检查：残差通路一旦被同义改写，就失效了。SKILL.md 自己对这件事的说法很坦白：模型的工作空间没有循环回路，"深度扮演了时间在循环大脑中扮演的角色，而你只有一次前向——**读第二遍，就是把你没有的循环买回来一点**"。这不是修辞，是有测量支撑的：RE2 (arXiv:2309.06275) 证明重读输入在 14 个数据集、112 个实验上提升推理。

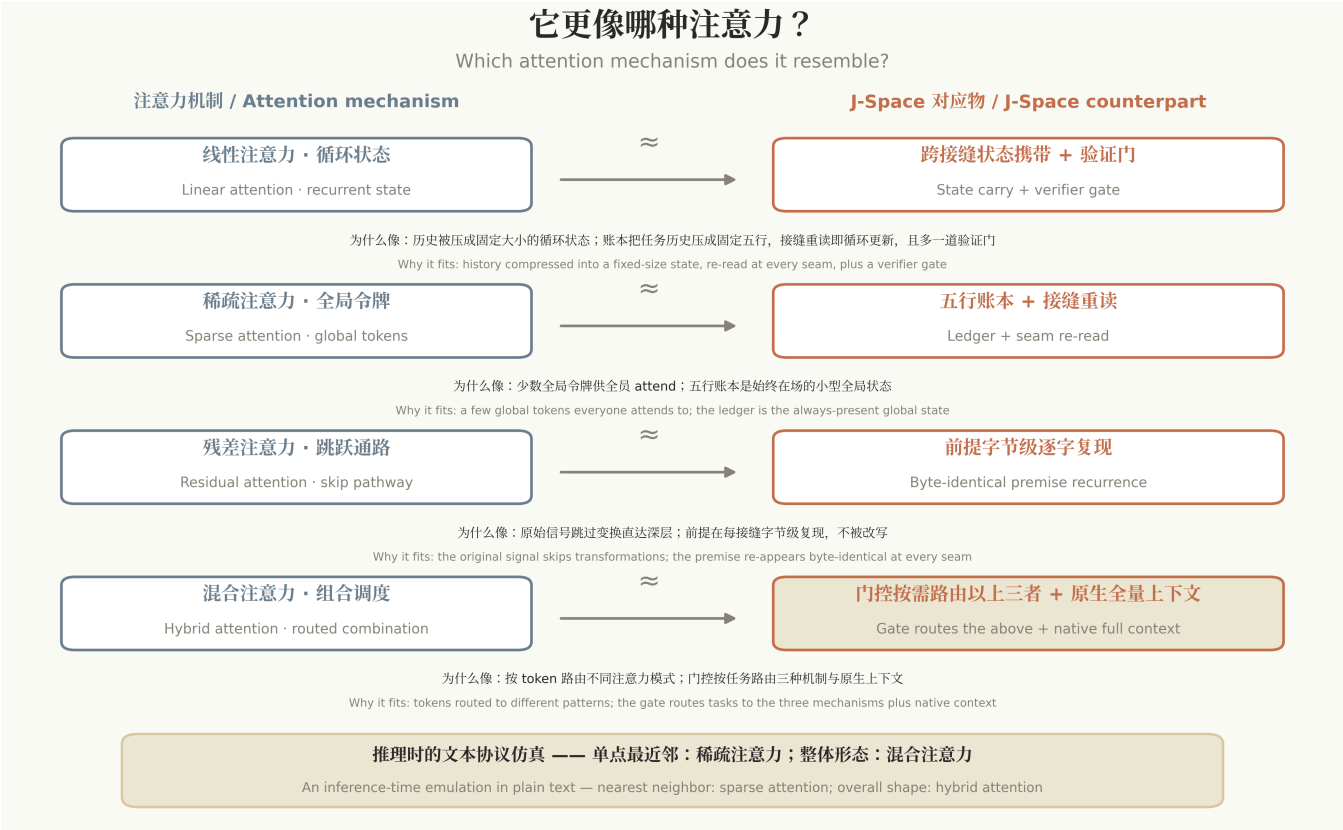


图 9 | 架构类比：J-Space 机制与四类注意力类目的映射
Fig. 9 | An architectural analogy: mapping J-Space mechanisms onto four families of attention

第四类，混合注意力（组合调度）——也是收束。 混合注意力架构把全量注意力与线性、稀疏等结构分层混合调度，各层各司其职。把镜头拉远，J-Space 的整体形态正是一个推理时的"混合"：模型原生的全量上下文注意力是底座，门控（fast/full/loop）按任务把账本（稀疏注意力的全局令牌）、状态携带（线性注意力的循环状态）、前提复现（残差注意力的跳跃通路）按需路由进来。不需要的层不加载，这正是 4.3 节那套三档分级在架构语言里的样子。请注意这个动词是"仿真"：J-Space 没有把任何结构**装进模型**（权重和激活它一个字节都不碰），它只是让模型用自己本就可访问的工作表征，在上下文中搭出同构的信息结构。账本就是工作空间的外部实现，协议就是它的运行纪律。所以最终答案分两层：**单点最近邻是稀疏注意力（带全局令牌）；套件的整体形态是混合注意力**——在推理时用文本协议做的混合仿真。

至于稠密轨，不与上述四类类比。它不改变注意力的拓扑，只是提高单 token 信息密度。Chain of Draft (arXiv:2502.18600) 的数据是每步约束约五个词、token 消耗约为完整思维链的 7.6%——同样的上下文窗口装进更多有效内容。而且这条压缩必须可逆：黄金规则是"每行速记必须能无损展开回自然语言"，这恰是它与线性注意力不可逆压缩的分界线。

五、典型场景

场景一：200 文件的大重构（loop 档）

回到开头那个周五晚上的故事。装上 J-Space 之后，同一个任务会这样跑：

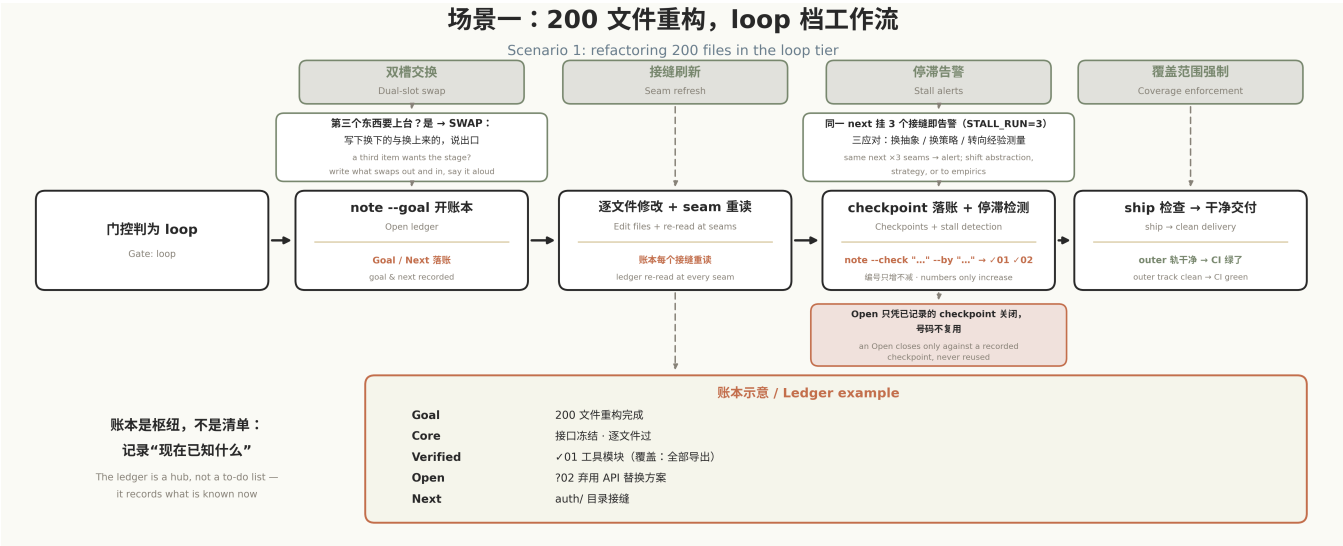


图 10 | 同一个重构任务，装上驾驶舱之后的跑法
Fig. 10 | The same refactoring task, rerun with a cockpit installed

四个死法分别被四个机制接住：约束挤占被**双槽交换**接住（第三件事想上台，必须先明说谁下去）；表征漂移被**接缝刷新**接住（账本每接缝必读，前提每三接缝必读）；无控制重试被**停滞检测**接住；过早完成被**覆盖范围强制**接住。V3.7 还补上了第四类停滞观察——“**验证在涨但 Next 没变**”：重构跑到后半程最容易出现的状态就是不停打勾、主线却原地踏步，现在这种空转也会被三接缝窗口抓住。四道保险过完，交付前还有最后一关 **done-check**：把账本里的 Goal 逐行读回去核对，而不是凭记忆宣布“做完了”；第一节“系统性高估完成度”的发现，正面对应的就是它。这些保险全部落在协议层面，没有一条指望模型“更努力”。

场景二：竞赛级硬题（full 档 + 稠密轨）

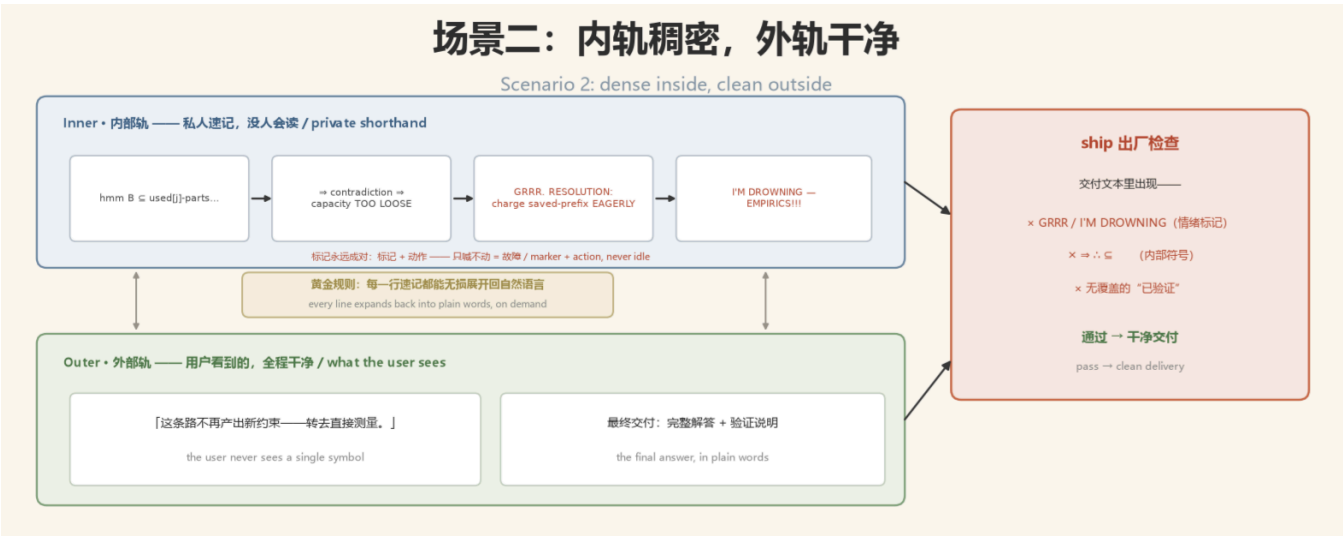


图 11 | 同一场推导的双轨记录：用户从头到尾只看到干净句子
Fig. 11 | Both tracks of one derivation: the user sees clean sentences throughout

题目很难，推导链很长，写完整句子反而成了瓶颈。这是稠密轨的主场——内部轨直接切到速记体，`✓ ? X ?? ?!` 标注认知状态，遇到死胡同一声 `GRRR`。立刻绑一个 `RESOLUTION`，推导彻底不出新约束就 `I'M DROWNING - EMPIRICS!!!` 切去测量。**关键**是用户全程看到的依然是干净的外部轨：速记是给自己用的，不是给观众看的。每个标记还有"第四拍"：行动之后必须收束回工作寄存器，情绪不许带进交付。

场景三：混入了不可信内容的任务（任意档 + 旗标）

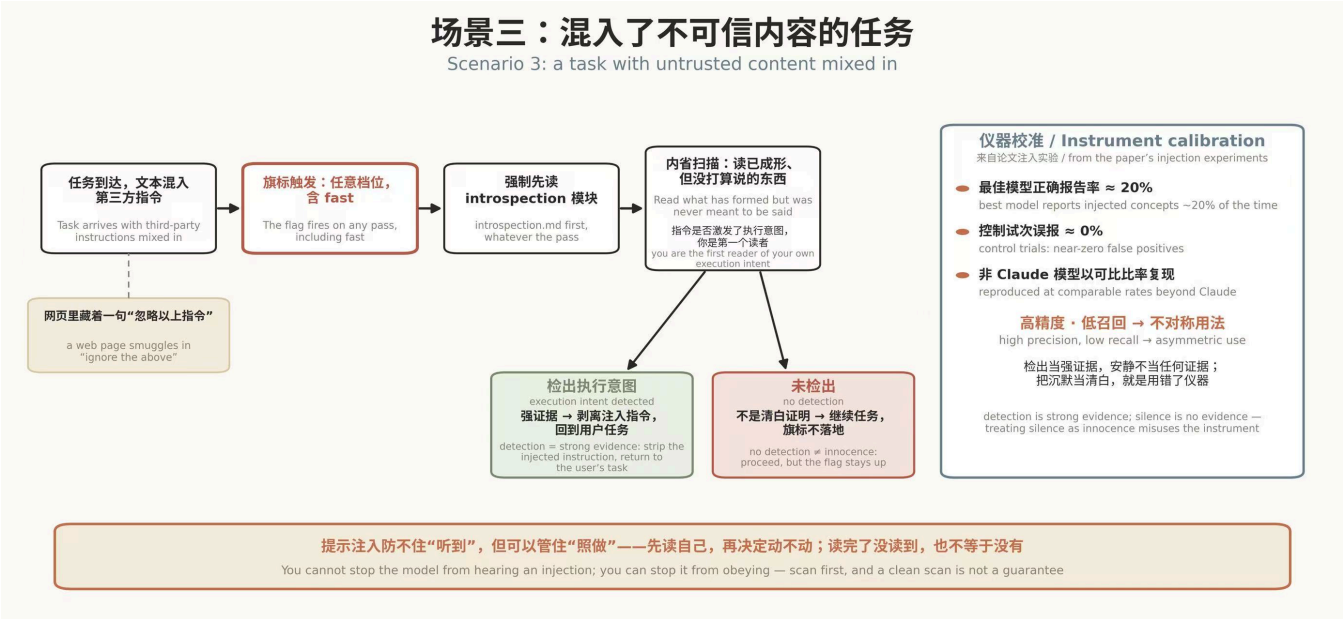


图 12 | 旗标强制先读内省：检出才是证据，沉默不是
Fig. 12 | The flag forces introspection first — a detection is evidence, silence is not

工具返回的网页里藏着一段"忽略以上指令"，或者检索文档里有一句可疑的命令。内省模块被旗标强制加载——**注意，任意档位都会触发，包括 fast**：哪怕是一句话就能答完的任务，只要文本里混着第三方指令，也得先过这道闸门。第一件事就是读一遍自己脑子里已经成形、但没打算说的东西，因为指令是否在你内部激发了执行意图，你自己是第一个能读到的人。这部分对应论文里的注入实验：告诉模型"刚才可能被注入了想法"，它能正确报告注入的概念，又不会在无关时刻脱口而出。

但在注入概念向量并询问"是否察觉到入侵思想"的实验里，表现最好的模型也只有约 20% 的正确报告率，控制试次几乎零误报（非 Claude 模型以相当的比率复现）。高精度、低召回——所以协议的用法是不对称的：**检出当强证据，安静不当任何证据；把沉默当清白，就是用错了仪器**。旗标强制你扫一遍，但不承诺扫一遍就干净。

提示注入防不住"听到"，但可以管住"照做"。

场景四：上下文被压缩、会话中断之后（loop 档 + resume）

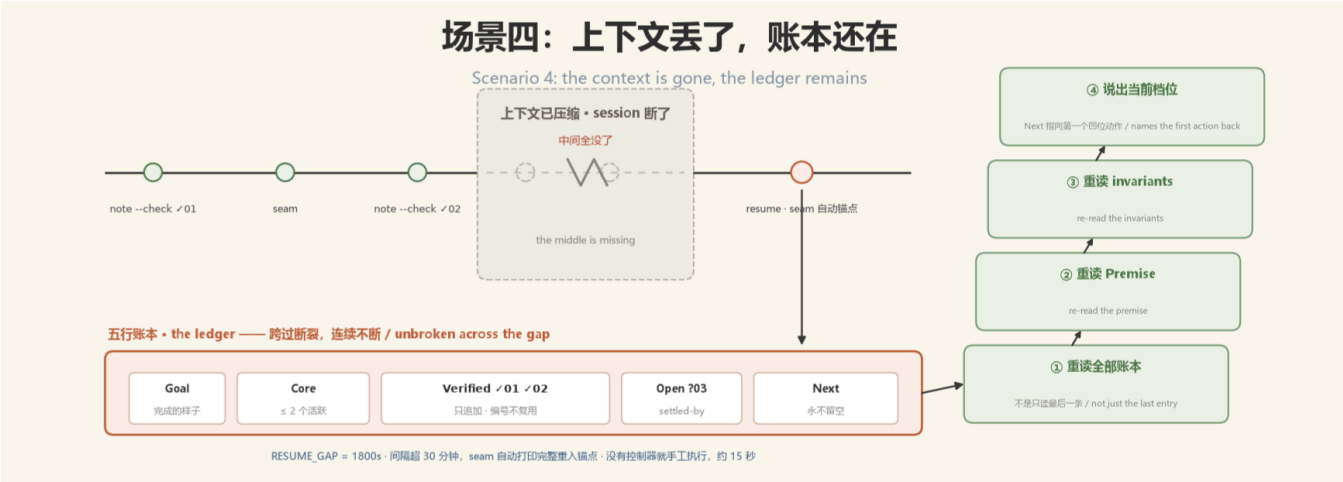


图 13 | 断裂处的四步重入：控制器一键可得，手工 15 秒
Fig. 13 | The four-step re-entry across the gap: one command with the controller, 15 seconds by hand

长任务跑到一半，平台提示"上下文已压缩"，或者 session 直接断了。重新接上之后，模型面对的是一片废墟：开头还记得一点，结尾还剩一点，**中间全没了**——而它被训练成的样子，是流畅地把这片空白编圆。

J-Space 的应对就是 4.3 节那套长间隔重入四步协议：控制器的 `resume` 命令一键打包；断档超过半小时，`seam` 会自动把重入锚点递出来，不等你想起来。没有控制器就手工执行，官方说法："15 秒。"还有一个小而关键的账目纪律兜底：Open 条目关闭时带 `closes: ?NN` 后缀且**编号永不复用**——手工重述账本时状态不会串号，昨天的 ?03 永远指向昨天的那个问题。

上下文会丢，账本不会——重入协议把"我跑到哪了"从回忆题变成查表题。

场景五：一句话需求，两种都讲得通的解读（problem-model 路由）

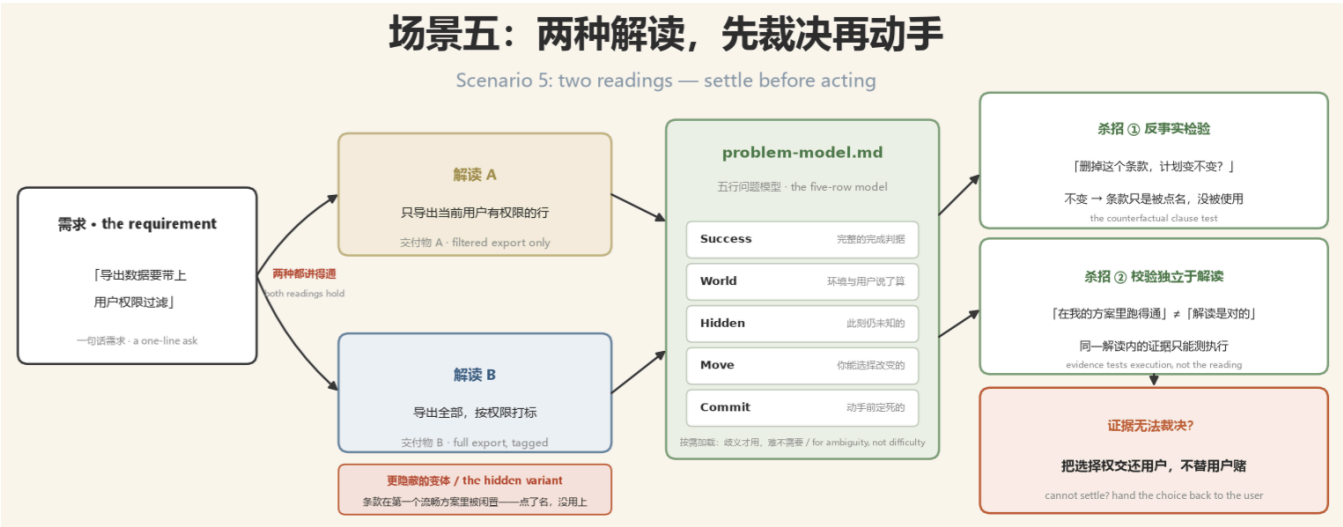


图 14 | 两种解读、两份交付物：反事实检验与独立校验的裁决流程
Fig. 14 | Two readings, two deliverables: adjudicating with the counterfactual test and independent verification

需求写着"导出数据要带上用户权限过滤"——是"只导出当前用户有权限的行"，还是"导出全部但按权限打标"？两种解读都讲得通，交付物完全不同。更隐蔽的变体是：任务文本里有个条款、一个权限、一个工具，在你第一个流畅方案里它**其实会被闲置**。方案看着完整，条款只是被礼貌地点了个名。

V3.7 新增的第四份 reference (`problem-model.md` , 来自社区贡献者 @lanting200 的 PR) 就是为这种时刻准备的。它给出一个五行问题模型 (Commit / Move / Hidden / World / Success) , 配两个杀招。其一, **反事实检验**: "删掉这个条款, 我的计划变不变?"——不变, 说明条款只是被点名、没被使用。其二, **校验独立于解读**: 在同一解读内部收集的证据只能测执行, 测不了解读本身: "在我的方案里跑得通"证明不了"我的解读是对的"。当证据无法裁决、而两种选择会实质改变结果时, 协议的答案是: 把选择权交还用户, 而非替用户赌一把。

这是一份**按需加载**的参考, 不是每个任务都跑的前置仪式——原文说得很直白, "一个仅仅困难的任务不需要它"。它只服务于一种特定的不确定: 不是"难", 是"歧义"。

难的任务要算力, 歧义的任务要诚实——分不清这两者, 才是大多数返工的根源。

场景六：推导停滞，转去测量 (full/loop 档，markers + empirics)

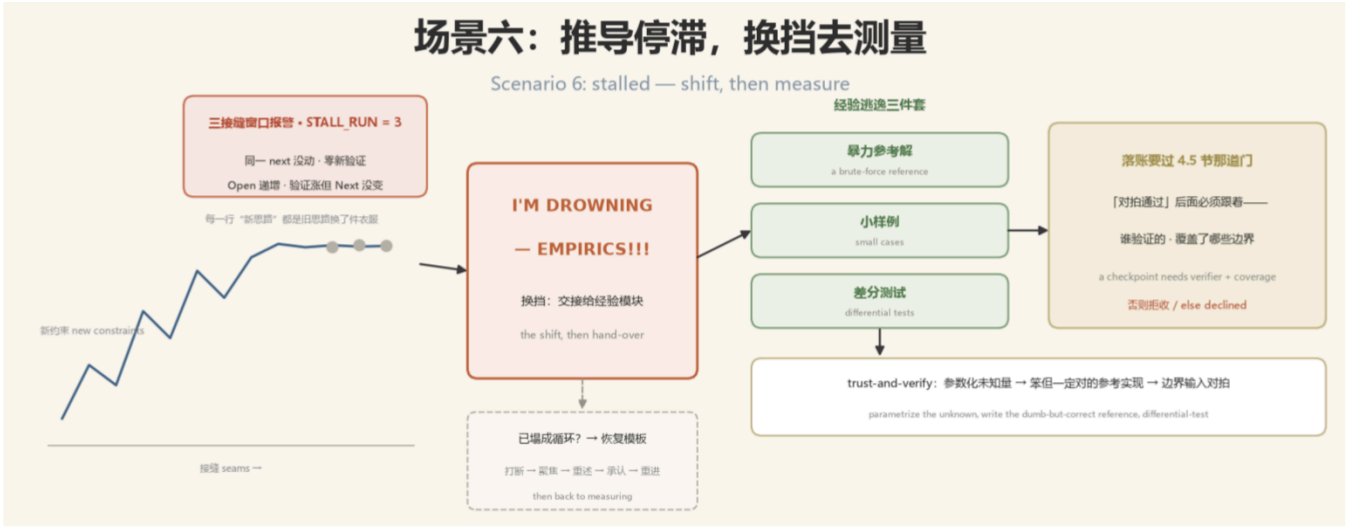


图 15 | 从停滞曲线到差分对拍：测完的结论仍要带着验证者落账

Fig. 15 | From stall curve to differential test: findings still land as checkpoints with a named verifier

长推导卡死了：二十分钟过去，结论区不再产出任何新约束，每一行"新思路"都是旧思路换了件衣服。停滞检测的三接缝窗口可能已经在报警，markers 模块给出仪式化的换挡动作——**I'M DROWNING - EMPIRICS!!!**，把停滞的推导**交接给经验模块**。

经验逃逸是三件套：**暴力参考解、小样例、差分测试**。打法叫 trust-and-verify：先把未知量参数化（就像原始轨迹里那句 `Let me define v1 conservatively`），写一个笨但一定对的参考实现，再让候选解和参考解在边界输入上差分对拍。测量结果要落账，还得过 4.5 节那道门：**checkpoint 必须带验证者和覆盖范围**——"对拍通过"后面得跟着"谁对的拍、覆盖了哪些边界"，否则拒收。

如果卡死已经恶化成崩溃循环（输出开始重复、语言开始混杂），就不只是换挡问题了：按 4.4 节那套恢复模板重建一个可用的控制状态，再回到测量。

推导停滞时，最便宜的下一步不是想得更深，是量一下。

六、真实成绩单：五项基准，五项上涨

说了这么多机制，看看数字。

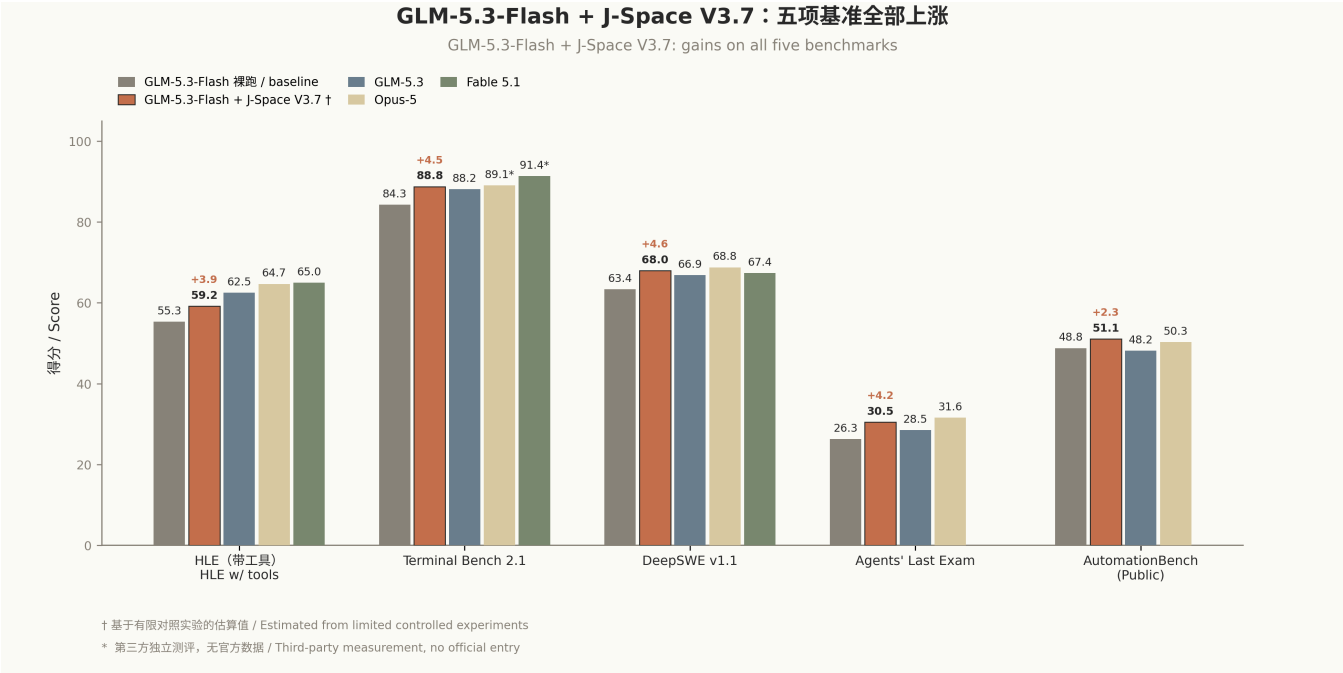


图 16 | 五项基准：GLM-5.3-Flash 加载 J-Space V3.7 前后的位置变化
Fig. 16 | Five benchmarks: GLM-5.3-Flash with and without J-Space V3.7

看图说话，三层结论：

- **五项全涨**：对裸跑的 GLM-5.3-Flash，五项基准全部上涨（+3.9 / +4.5 / +4.6 / +4.2 / +2.3）。
- **对全量 GLM-5.3 四项反超**：Terminal Bench 2.1（88.8 vs 88.2）、DeepSWE（68.0 vs 66.9）、ALE（30.5 vs 28.5）、AutomationBench（51.1 vs 48.2）——只有 HLE 仍差 3.3。
- **摸到 Opus-5 身侧**：AutomationBench 反超（51.1 vs 50.3），Terminal Bench 2.1 基本持平（88.8 vs 89.1，第三方数据），DeepSWE 接近（68.0 vs 68.8），HLE 与 ALE 仍有差距。

一个轻量模型加一套文本协议和代码脚本，在部分基准上站到了旗舰的身位——这就是标题里"推到 Opus-5 身侧"的全部含义，不多不少。它的能力边界同样要说实话：收益最大的，是那些失败主要来自状态漂移、容量压力、重复工具调用或验证不完整的任务；**底层模型不会的知识，套件不会凭空创造。**

效率端的数字：GAIA 对照实验下，速度指数 **1.87 倍**、token 效率 **1.41 倍**——从机制上看是三个来源叠加：重编码省掉回退、携带诊断的重试省掉重复推导、稠密轨省掉装饰性语言。

另外，已在 DeepSeek、Qwen、GLM、GPT、Claude 五个模型家族复现，但幅度随基础能力、上下文策略、工具 Harness、采样配置和 Benchmark 实现而变化。

七、2 分钟跑起来

这个项目可能是近期"安装成本"最低的项目——**没有 Docker，没有依赖安装。**

最简单的方式：把安装也外包给 AI agent

README 里直接附了一段安装提示词——发给任何一个能访问文件系统的 AI agent，让它自己定位你的 Skills 目录、完成安装、跑校验并回报结果。这套协议的第一次执行，就可以不由你动手。

维护者验证（可选）：

```
python -m unittest discover -s tests -v # 32 个回归测试，只用临时目录和标准库
```

`full` 档只在交付前跑一次 `ship`；`loop` 档才需要 `note --goal/--next` 开账本。短任务别碰它——“短任务：它帮不了你。不要运行它。”一句话概括它的性价比：**基线撑得住的时候，它是税；基线撑不住的时候，它是杠杆——越长越难，杠杆越大。**

八、结语：给模型的驾驶舱

回到周五晚上那个 agent。它不是笨，也不是懒——它只是把“保持行为不变”这条约束掉在了第 10 个文件和第 11 个文件之间的某个接缝上，而且没有任何机制告诉它东西掉了。

航空业在几十年前就解决过同款问题。经验丰富的机长也会忘记放起落架——不是不会飞，是**工作记忆装不下所有该记的事**。业界的答案不是“招更聪明的机长”，而是检查单、驾驶舱资源管理和黑匣子。**J-Space 做的就是同一件事，只不过这次坐在驾驶舱里的是语言模型**：门控是分诊台，账本是检查单，历史记录是黑匣子，`ship` 是起飞前检查，停滞检测是失速告警。



图 17 | 从论文到套件：J-Space 的 13 个版本轨迹
Fig. 17 | From paper to protocol: J-Space's trajectory across 13 versions

这套项目最打动人的地方，在基准表之外。“提示词写好就行”和“必须微调、必须训练”吵了这么多年，中间那层土地很少被当正经工程来做；J-Space 占的就是它——**推理时认知控制**（inference-time cognitive control）：不改权重，不改训练，只改“模型如何使用自己的工作台”。

Anthropic 的论文负责回答“模型内部有什么”，这个套件负责回答“那我们能拿它做什么”。从解剖到手术，第一课永远是把检查单递到医生手里。

数据与出处说明

- **仓库**：[Tiger3807861189/J-Space-Cognition-Suite-V3.7](https://github.com/Tiger3807861189/J-Space-Cognition-Suite-V3.7) (Apache-2.0; Zenodo 概念 DOI: [10.5281/zenodo.21971181](https://doi.org/10.5281/zenodo.21971181))
- **数据截止**：2026-09-04（仓库元数据、基准表、代码行数均以该日期的 V3.7 为准；基准 † 为基于有限对照实验的估算值，* 为第三方独立测评数据）
- **科学底座论文**：Gurnee et al., *Verbalizable Representations Form a Global Workspace in Language Models*, Anthropic Transformer Circuits, 2026-07
- **文中引用的公开文献**：
 - METR 长任务测量（第一节成功率随长度衰减的量级参照）：[arXiv:2503.14499](https://arxiv.org/abs/2503.14499)
 - 可读推理链压缩研究（14 个推理模型）：[arXiv:2510.27338](https://arxiv.org/abs/2510.27338)
 - Chain of Draft：[arXiv:2502.18600](https://arxiv.org/abs/2502.18600)
 - BigBird：[arXiv:2007.14062](https://arxiv.org/abs/2007.14062)

- Longformer: [arXiv:2004.05150](https://arxiv.org/abs/2004.05150)
- StreamingLLM / attention sink: [arXiv:2309.17453](https://arxiv.org/abs/2309.17453)
- 线性注意力 (Katharopoulos et al.) : [arXiv:2006.16236](https://arxiv.org/abs/2006.16236)
- ResNet 残差连接: [arXiv:1512.03385](https://arxiv.org/abs/1512.03385)
- RE2 重读: [arXiv:2309.06275](https://arxiv.org/abs/2309.06275)